به نام خدا



دانشکده ریاضی و آمار

اصول سیستم‌های عامل
نیم‌سال اول ۰۴-۰۵
استاد: دکتر رفیعی
نام دانشجو - شماره دانشجویی

تمرین سری پنجم

۱. به سوالات زیر پاسخ دهید.

(آ) تفاوت اصلی فرآیندهای مستقل (Independent) و همکار (Cooperating) چیست؟

(ب) سه چالش اصلی در ارتباط فرآیندهای همکار را نام ببرید.

(ج) شرط انحصار متقابل (Mutual Exclusion) در دسترسی به بافر مشترک به چه معناست؟

(د) بخش باقی‌مانده (Remainder Section) در ساختار حل مسئله ناحیه بحرانی شامل چه کدهایی است؟

(ه) چگونه اجرای هم‌روند دستورات ساده‌ای مانند `count++` و `count--` در سطح زبان ماشین می‌تواند منجر به ناهماهنگی داده‌ها شود؟

پاسخ:

(آ) فرآیندهای مستقل داده مشترک ندارند و بر یکدیگر تاثیر نمی‌گذارند، اما فرآیندهای همکار می‌توانند از فرآیندهای دیگر تاثیر بپذیرند یا بر اجرای آن‌ها اثر بگذارند.

(ب) این چالش‌ها شامل همگام‌سازی (Synchronization)، تبادل اطلاعات (Communication) و رقابت (Race) هستند.

(ج) بدین معناست که در هر لحظه تنها یک فرآیند باید توانایی دسترسی به بافر مشترک و تغییر آن را داشته باشد.

(د) به تمامی کدهای فرآیند که خارج از سه بخش اصلی ورود (Entry)، خروج (Exit) و ناحیه بحرانی (Critical Section) قرار دارند، بخش باقی‌مانده اطلاق می‌شود.

(ه) این دستورات در سطح ماشین به چندین دستورالعمل مجزا (مانند بارگذاری در رجیستر، انجام محاسبات و ذخیره مجدد) تقسیم می‌شوند. در صورت وقوع تعویض متن (Context Switch) در میان این مراحل، ممکن است مقدار نهایی ذخیره شده در حافظه بر اساس مقادیر قدیمی رجیسترها بازنویسی شود و نتیجه‌ای ناصحیح (مثلاً ۴ یا ۶ به جای ۵) حاصل گردد.

۲. (آ) سه نیازمندی اساسی برای یک راه حل معتبر در مسئله ناحیه بحرانی را نام ببرید.

(ب) چرا در تلاش اول دکر ،(Decker) ویژگی «پیشرفت» نقض می‌شود؟

(ج) مشکل اصلی تلاش دوم دکر که منجر به نقض انحصار متقابل می‌شود چیست؟

(د) تفاوت مفهومی بن‌بست (Deadlock) و قفل زنده (Livelock) در چیست؟

(ه) رابطه منطقی بین پدیده گرسنگی ،(Starvation) بن‌بست (Deadlock) و ویژگی انتظار محدود را تحلیل کنید.

پاسخ:

(آ) این سه نیازمندی عبارتند از: انحصار متقابل (Mutual Exclusion)، پیشرفت (Progress) و انتظار محدود (Bounded Waiting).

(ب) چون ورود به ناحیه بحرانی به صورت نوبتی اجباری شده است؛ اگر فرآیندی که نوبتش است در ناحیه غیربحرانی بماند، فرآیند دیگر حتی در صورت نیاز، اجازه ورود به ناحیه بحرانی را نخواهد داشت.

(ج) مشکل در اینجاست که بررسی وضعیت پرچم (flag) و تغییر آن به صورت اتمیک انجام نمی‌شود؛ در نتیجه ممکن است هر دو فرآیند هم‌زمان پرچم دیگری را FALSE مشاهده کرده و هر دو وارد ناحیه بحرانی شوند.

(د) در بن‌بست فرآیندها به طور ابدی متوقف شده و منتظر می‌مانند، اما در قفل زنده فرآیندها مدام تغییر وضعیت می‌دهند ولی به دلیل عدم هماهنگی در سرعت نسبی، عملاً پیشرفتی حاصل نمی‌شود.

(ه) طبق منابع، اگر در سیستمی تضمین شود که گرسنگی رخ نمی‌دهد ($\neg Starvation$)، می‌توان نتیجه گرفت که بن‌بست و قفل زنده نیز هرگز رخ نخواهند داد. در واقع، عدم وقوع گرسنگی به معنای برقراری شرط انتظار محدود است که سلامت پروتکل را از نظر زنده بودن (Liveness) تضمین می‌کند.

۳. (آ) مشکل اصلی «تلاش چهارم الگوریتم دکر» چیست و در تلاش پنجم چگونه برطرف شده است؟

(ب) در الگوریتم پترسون (برای دو فرآیند)، متغیرهای flag و turn چه نقش متفاوتی ایفا می‌کنند؟

(ج) چرا روش سخت‌افزاری «ازکار انداختن وقفه» (Disable Interrupts) برای سیستم‌های چندپردازنده‌ای مناسب نیست؟

(د) دستورالعمل TSL (Test and Set Lock) چگونه کار می‌کند و کدام یک از سه شرط اصلی مسئله ناحیه بحرانی را ارضا نمی‌کند؟

پاسخ:

(آ) مشکل اصلی تلاش چهارم، وقوع قفل زنده (Livelock) است. در این سناریو، فرآیندها وارد یک سیکل معیوب از «ادب و تعارف» می‌شوند؛ هر دو پرچم (flag) خود را پایین می‌آورند تا به دیگری اجازه دهند و دوباره هم‌زمان آن را بالا می‌برند، در نتیجه هیچ‌کدام وارد ناحیه بحرانی نمی‌شوند. در تلاش پنجم، دکر متغیر turn (نوبت) را با flag ترکیب کرد. در این حالت، اگر هر دو فرآیند هم‌زمان درخواست ورود داشته باشند، تنها فرآیندی که نوبت (turn) با اوست پافشاری می‌کند و فرآیند دیگر وارد فاز انتظار می‌شود تا نوبت به او برسد. این کار هم‌زمانی انحصار متقابل، پیشرفت و انتظار محدود را تضمین می‌کند.

(ب) در الگوریتم پترسون:

- متغیر flag نشان‌دهنده تمایل و آمادگی فرآیند برای ورود به ناحیه بحرانی است.
- متغیر turn تعیین‌کننده حق تقدم است. هر فرآیند پس از اعلام آمادگی، با تنظیم turn به مقدار شناسه فرآیند دیگر، با «ادب» نوبت را به دیگری تعارف می‌کند. اگر هر دو هم‌زمان درخواست دهند، فرآیندی وارد می‌شود که turn به نام او نیست (یا به عبارتی، فرآیندی که دیرتر turn را تغییر داده است، نوبت را به دیگری داده و خودش منتظر می‌ماند).

(ج) زیرا در سیستم‌های چندپردازنده‌ای، غیرفعال کردن وقفه تنها روی پردازنده‌ای که دستور را اجرا کرده اثر می‌گذارد. سایر فرآیندها که روی پردازنده‌های دیگر در حال اجرا هستند، همچنان می‌توانند به ناحیه بحرانی دسترسی داشته باشند و شرط انحصار متقابل نقض می‌شود. همچنین اگر فرآیندی در ناحیه بحرانی از کار بیفتد، کل سیستم (به دلیل غیرفعال ماندن وقفه‌ها) دچار اختلال می‌شود.

(د) دستور TSL به صورت اتمیک (غیرقابل تقسیم) محتویات کلمه حافظه (Lock) را می‌خواند و مقدار آن را به ۱ تغییر می‌دهد. اگر مقدار خوانده شده ۰ باشد، فرآیند وارد ناحیه بحرانی می‌شود؛ اگر ۱ باشد، در حلقه انتظار مشغول می‌ماند. این روش شرط انتظار محدود (Bounded Waiting) را ارضا نمی‌کند. زیرا اگر چند فرآیند منتظر باشند و فرآیند داخل ناحیه بحرانی خارج شود، انتخاب فرآیند بعدی تصادفی یا وابسته به سخت‌افزار است و ممکن است یک فرآیند دچار گرسنگی شده و تا ابد منتظر بماند.

۴. با مقایسه الگوریتم پترسون (راهکار نرم‌افزاری) و دستورالعمل TSL (راهکار سخت‌افزاری)، هر دو از مکانیسم «انتظار مشغول» (Busy Waiting) استفاده می‌کنند. با این حال، چرا گفته می‌شود الگوریتم پترسون شرط «انتظار محدود» را برآورده می‌کند، اما TSL ممکن است باعث گرسنگی (Starvation) شود؟ (تحلیل کنید چه مکانیزمی در پترسون وجود دارد که در TSL غایب است).

پاسخ:

اگرچه هر دو روش از انتظار مشغول (while loop) استفاده می‌کنند، تفاوت بنیادین در نحوه مدیریت نوبت است:

- در الگوریتم پترسون: یک متغیر صریح به نام turn وجود دارد که ترتیب ورود را کنترل می‌کند. وقتی فرآیندی از ناحیه بحرانی خارج می‌شود یا قصد ورود دارد، وضعیت متغیرهای flag و turn به گونه‌ای تغییر می‌کند که فرآیند دیگر که در حلقه انتظار گیر افتاده است، شرط حلقه‌اش (while) نقض شده و حتماً اجازه ورود پیدا می‌کند. این ساختار منطقی تضمین می‌کند که اگر فرآیند P۰ منتظر باشد، P۱ نمی‌تواند چندین بار پشت سر هم وارد ناحیه بحرانی شود و نوبت نهایتاً به P۰ می‌رسد (جلوگیری از گرسنگی).
- در دستورالعمل :TSL هیچ متغیری برای حفظ «نوبت» یا «صف» وجود ندارد. تمام فرآیندهای منتظر، دائماً در حال تلاش برای اجرای دستور TSL هستند تا قفل (lock) را بگیرند. وقتی قفل آزاد می‌شود (صفر می‌شود)، اینکه کدام فرآیند زودتر دستور TSL را اجرا کند و قفل را دوباره یک کند، کاملاً وابسته به سرعت پردازنده و زمان‌بندی سخت‌افزاری است. ممکن است یک فرآیند سریع‌تر همیشه برنده شود و فرآیند دیگر دائماً در رقابت شکست بخورد و دچار گرسنگی شود.

۵. قفل‌های mutex که بر پایه «انتظار مشغول» (busy-waiting) هستند و با نام spin-lock شناخته می‌شوند، در چه شرایطی نسبت به قفل‌های مبتنی بر «مسدود کردن» (blocking) ترجیح داده می‌شوند؟

پاسخ:
قفل‌های spin-lock زمانی گزینه بهتری هستند که **زمان انتظار برای قفل بسیار کوتاه باشد.** دلیل این امر آن است که مسدود کردن یک فرآیند و بیدار کردن مجدد آن مستلزم انجام **دو تعویض متن** (یکی برای به خواب رفتن و دیگری برای بیدار شدن) است که هزینه زمانی و پردازشی بالایی دارد. اگر زمان لازم برای چرخش (spinning) در انتظار قفل، کمتر از زمان مورد نیاز برای تعویض متن باشد، استفاده از spin-lock به صرفه‌تر خواهد بود. این قفل‌ها به‌ویژه در **سیستم‌های چند هسته‌ای** مفید هستند، زیرا یک هسته می‌تواند در حال اجرای بخش بحرانی باشد و هسته دیگر بدون نیاز به تعویض متن، برای مدتی کوتاه منتظر آزاد شدن آن بماند.

۶. فرض کنید در کد زیر p1 و p2 همروند باشند و سمافورهای x و y دو سمافور دودویی و هر دو مقدار اولیه یک را داشته باشند. همچنین فرض کنید دستور print در یک چرخه دستورالعمل اجرا شود.

```
p2:                          p1:
----                         ----
wait(y)                      wait(x)
while true do                while true do
print "B"                    print "A"
signal(y)                    signal(x)
----                         ----

```

آیا هر تلفیقی از A ها و B ها ممکن است در خروجی ظاهر شوند؟ توضیح دهید.

پاسخ:
با توجه به اینکه این دو فرآیند از سمافورهای متفاوتی استفاده می‌کنند هیچ کدام منتظر دیگری نمی‌ماند. با فرض اینکه این دو فرآیند همروند اجرا شوند، بنابراین در هر قسمتی از اجرا ممکن است اجرای آن‌ها متوقف شود و CPU به فرآیند دیگر اختصاص داده شود که این بدان معناست که خروجی این برنامه‌ها از الگوی خاصی پیروی نمی‌کند و هر حالتی ممکن است داشته باشد.

۷. ساختار monitor چه مشکلاتی را حل می‌کند که قفل‌های mutex و monitor با آنها دسته پنجه نرم می‌کنند؟ این کار را به چه شکلی انجام می‌دهد؟

پاسخ:
استفاده مستقیم از سمافورها مستعد **خطاهای منطقی و زمانی** است؛ برای مثال، جابجا نوشتن ترتیب wait و signal، فراموش کردن یکی از آن‌ها، یا استفاده مکرر از wait روی یک سمافور که می‌تواند منجر به نقض انحصار متقابل یا ایجاد **بن‌بست (deadlock)** دائمی شود. monitor این مشکل را با ارائه یک **نوع داده انتزاعی (ADT)** حل می‌کند. مانیتور متغیرهای مشترک و توابع دسترسی به آن‌ها را در یک بسته واحد کپسوله می‌کند و **تضمین می‌کند که در هر لحظه تنها یک فرآیند در داخل مانیتور فعال باشد.** با این مکانیزم، برنامه‌نویس نیازی به کدنویسی صریح برای مدیریت قفل‌ها ندارد و احتمال خطاهای انسانی به شدت کاهش می‌یابد.

۸. (آ) نحوه همگام‌سازی در مسئله تولیدکننده–مصرف‌کننده را با استفاده از مانیتور شرح دهید.
(ب) یک نمونه مانیتور برای حل مسئله «غذا خوردن فیلسوف‌ها» بنویسید که از بن‌بست جلوگیری کند.

پاسخ:

(آ) در راهکار مانیتور برای تولیدکننده–مصرف‌کننده، از دو متغیر شرطی (condition variables) به نام‌های full و empty استفاده می‌شود. اگر بافر پر باشد، تولیدکننده با دستور full.wait() معلق می‌شود. وقتی مصرف‌کننده آیتمی را برمی‌دارد، با full.signal() به تولیدکننده اطلاع می‌دهد. به همین ترتیب، اگر بافر خالی باشد، مصرف‌کننده روی empty.wait() منتظر می‌ماند تا تولیدکننده پس از درج آیتم جدید، empty.signal() را فراخوانی کند.

(ب) نمونه مانیتور:

```
monitor DiningPhilosophers {
    enum {THINKING, HUNGRY, EATING} state;
    condition self;

    function pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait();
    }

    function putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5); // Check left neighbor
        test((i + 1) % 5); // Check right neighbor
    }

    function test(int i) {
        if ((state[(i + 4) % 5] != EATING)
            && (state[i] == HUNGRY)
            && (state[(i + 1) % 5] != EATING))
        {
            state[i] = EATING;
            self[i].signal();
        }
    }

    initialization_code() {
        for (int i = 0; i < 5; i++) state[i] = THINKING;
    }
}
```